FINAL PROJECT 결과발표

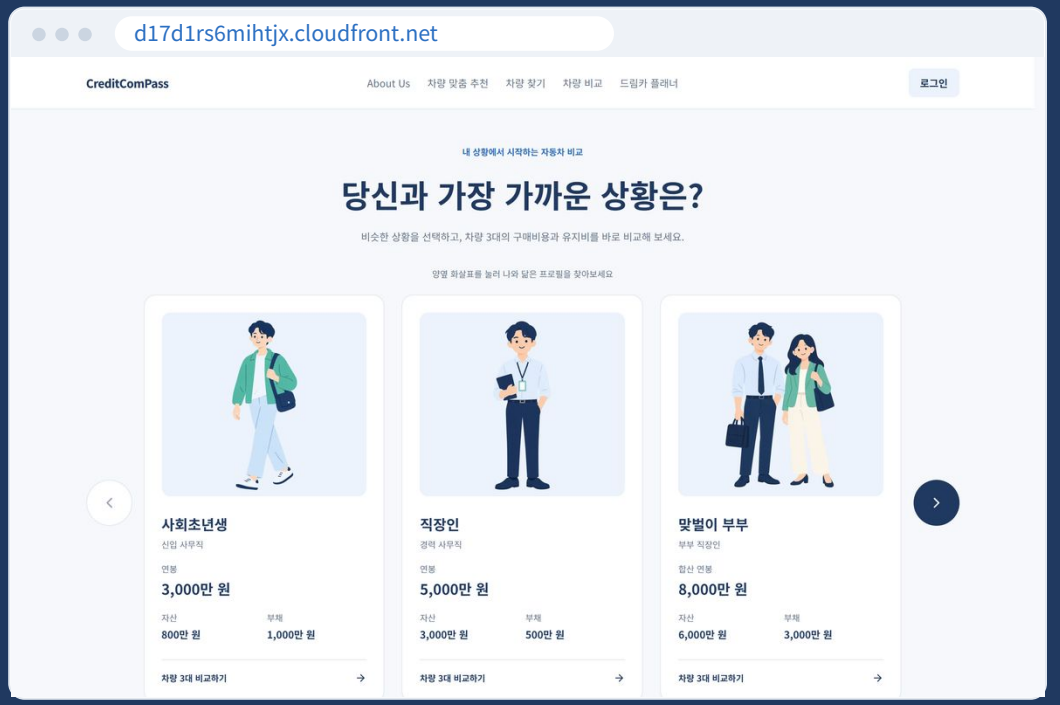
# CreditComPass

재무 상황과 차량 취향을 연결하는
자동차 구매 의사결정 서비스

2조 인생은 코스피처럼

이원준, 하형철, 류상민, 지승렬

▶ 실습 사이트 바로가기



02 CONTENTS

# 목차


01 **팀 소개**
팀원과 맡은 역할

02 **프로젝트 소개 및 기획 의도**
사용자 문제, 기획 의도, 요구사항, 화면 변화

03 **주요 기능** 실습
진단, 추천, 비교, 플래너, 콤파, 운영

04 **시스템 아키텍처**
React, Spring Boot, FastAPI 구성과 저장 위치

05 **데이터 파이프라인**
수집 → 정제 → DB 적재

06 **데이터 분석 및 모델링**
데이터 탐색, 모델 구조, 평가, 후속 실험

07 **ERD**
차량 원본과 실제 저장 구조

08 **보안 적용 내용**
로그인, 권한, 입력 검증, 자동 가입 방지

09 **협업 방식**
브랜치 전략과 일정 관리

10 **트러블슈팅**
주요 문제 해결 사례

11 **향후 개선 방향**
개발 결과, 한계, 다음 과제


▶ **실습 사이트**
d17d1rs6mihtjx.cloudfront.net

# 팀 소개


**01 팀 소개**
팀원과 맡은 역할

02 프로젝트 소개 및 기획 의도
사용자 문제, 기획 의도, 요구사항, 화면 변화

03 주요 기능 실습
진단, 추천, 비교, 플래너, 콤파, 운영

04 시스템 아키텍처
React, Spring Boot, FastAPI 구성과 저장 위치

05 데이터 파이프라인
수집 → 정제 → DB 적재

06 데이터 분석 및 모델링
데이터 탐색, 모델 구조, 평가, 후속 실험

07 ERD
차량 원본과 실제 저장 구조

08 보안 적용 내용
로그인, 권한, 입력 검증, 자동 가입 방지

09 협업 방식
브랜치 전략과 일정 관리

10 트러블슈팅
주요 문제 해결 사례

11 향후 개선 방향
개발 결과, 한계, 다음 과제


**▶ 실습 사이트**
d17d1rs6mihtjx.cloudfront.net

04 TEAM

# 팀 소개와 담당 영역

2조 인생은 코스피처럼

## 이

### 이원준

- 재무 진단
- 차량 UX
- 배포

## 하

### 하형철

- 데이터
- 추천 모델
- 모델 고도화

## 류

### 류상민

- 인증
- 회원
- 상담

## 지

### 지승렬

- 관리자
- 연동
- Agent 파일 추가
- 방향성 체크

**공통 협업** GitHub로 코드를 합치고, API 규칙과 차량 ID, 데이터 기준을 함께 맞췄습니다

# 프로젝트 소개 및 기획 의도


01 팀 소개
팀원과 맡은 역할

02 프로젝트 소개 및 기획 의도
사용자 문제, 기획 의도, 요구사항, 화면 변화

03 주요 기능 실습
진단, 추천, 비교, 플래너, 콤파, 운영

04 시스템 아키텍처
React, Spring Boot, FastAPI 구성과 저장 위치

05 데이터 파이프라인
수집 → 정제 → DB 적재

06 데이터 분석 및 모델링
데이터 탐색, 모델 구조, 평가, 후속 실험

07 ERD
차량 원본과 실제 저장 구조

08 보안 적용 내용
로그인, 권한, 입력 검증, 자동 가입 방지

09 협업 방식
브랜치 전략과 일정 관리

10 트러블슈팅
주요 문제 해결 사례

11 향후 개선 방향
개발 결과, 한계, 다음 과제


▶ 실습 사이트
d17d1rs6mihtjx.cloudfront.net

06 PROBLEM

# “내가 원하는 차를, 지금의 생활 예산으로 감당할 수 있을까?”

| 사용자가 겪는 문제 | 서비스의 대응 |
| --- | --- |
| 1 월 소득이 같아도 지출, 기존 대출, 모아 둔 돈은 사람마다 다릅니다 | → 소득, 지출, 상환액, 목돈을 물어보고 내 차량 예산을 정리합니다 |
| 2 차 값과 매달 드는 유지비를 함께 봐야 합니다 | → 차량 가격 옆에 월 할부금과 예상 월 유지비를 함께 보여줍니다 |
| 3 조건을 바꾸면 비용과 적합도가 어떻게 달라지는지 알고 싶습니다 | → 최대 3대를 같은 조건으로 비교하고 구매 조건을 바꿔 봅니다 |

07 PLANNING INTENT

# 기획 의도와 비슷한 서비스와의 차이

기획 발표 때 조사한 비슷한 서비스는 대부분 차량 탐색과 구매 지원에 집중합니다

### 엔카
차량 탐색, 구매 지원

### KB차차차
차량 탐색, 구매 지원

### K Car
차량 탐색, 구매 지원

CreditComPass가 이어 주는 흐름

STEP 1
**나의 재무 조건**
→
STEP 2
**적격 차량 후보**
→
STEP 3
**취향에 따른 순위**
→
STEP 4
**비용 비교**
→
STEP 5
**준비계획**

초점 거래나 대출 중개보다 **구매 판단을 돕는 과정**에 집중했습니다

08 PLAN vs NOW

# 처음 기획과 지금 구현은 이렇게 달라졌습니다

| 항목 | 기획발표 기준 | 현재 구현 기준 |
| --- | --- | --- |
| **분석 목표** | 신용위험 분류 모델을 만들어 추천에 반영 | **재무 적격 조건과 차량 취향을 반영한 차량 순위 모델** |
| **첫 진입** | 재무 진단 중심 | **상황별 가상 프로필로 차량 3대를 비교해 보고, 직접 진단도 가능** |
| **입력 화면** | 소득, 자산, 신용, 차량, 확인 순서로 구상 | **재무 문답 → 이용 환경 → 차량 취향 → 확인** |
| **이용 권한** | 종합 분석 뒤 추천 단계부터 로그인 필요 | **비회원도 추천 가능, 기록과 개인 기능은 로그인 필요** |
| **차량 원본** | brands, vehicles 등 초기 테이블 | **managed_vehicles 하나로 통합** |
| **계획 저장** | 서버 테이블에 저장하도록 설계 | **진단 요약은 서버에, 관심 차량과 드림카 계획은 브라우저에 저장** |
| **서비스 범위** | 추천, 비교, 시뮬레이션, 계획 | **취향 테스트, 콤파, 1:1 상담, 운영 관리 추가** |

09 REQUIREMENTS

# 사용자 요구를 어떤 기능으로 풀었나

| 사용자 요구 | 현재 대응하는 기능 | 발표 증거 |
| --- | --- | --- |
| **긴 입력 전에 서비스를 이해하고 싶다** | 가상 프로필 비교 체험 | 메인 → 비교 화면 |
| **내가 쓸 수 있는 예산을 알고 싶다** | 소득, 지출, 상환액, 목돈 입력과 예산 요약 | 재무 입력과 분석 화면 |
| **내 상황과 취향을 반영하고 싶다** | 이용 환경, 좌석, 차체, 연료, 취향과 중요도 | 설문과 추천 요청 구조 |
| **비슷한 후보를 같은 기준으로 비교하고 싶다** | 최대 3대 비교, 월 비용과 재무 적합도 | 비교 화면 |
| **원하는 차를 위해 준비하고 싶다** | 드림카 자금과 저축 계획 | 플래너와 진행 현황 |
| **어려운 설명을 이해하고 싶다** | 용어 안내, 콤파, 1:1 상담 | 도움말과 상담 화면 |
| **운영자가 정보를 관리해야 한다** | 차량, 모델, 회원, 권한, 이력 관리 | 관리자 화면 |

**품질 요구** 입력 검증 · 회원 범위 분리 · 데이터와 모델 버전 일치 · 실패 시 입력 유지

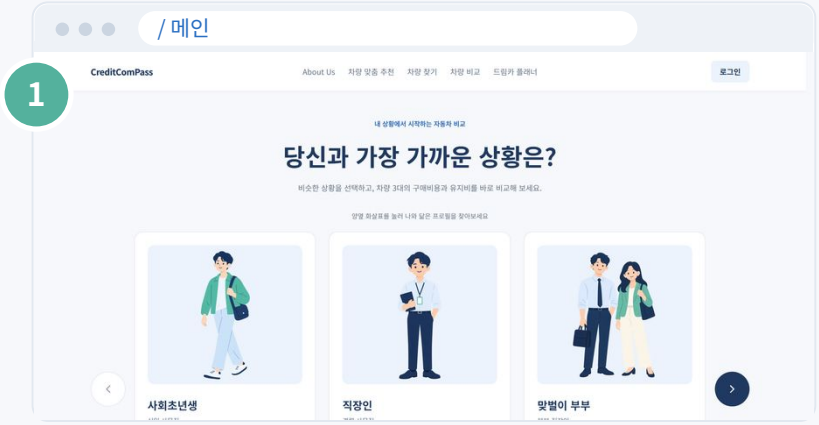
# 화면은 시작 방식과 입력 방식이 달라졌습니다

### 초기 설계

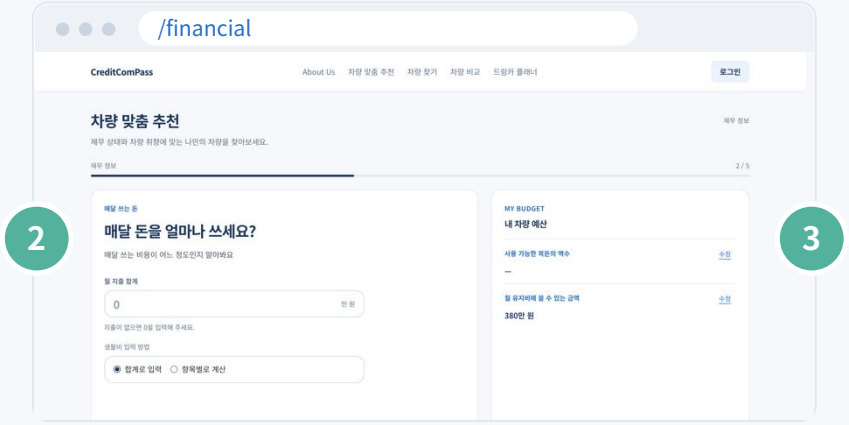
- Figma로 그린 초기 메인과 재무, 자산, 신용 입력 화면
- 진단 위주로 순서대로 설명하는 구성

### 현재 화면

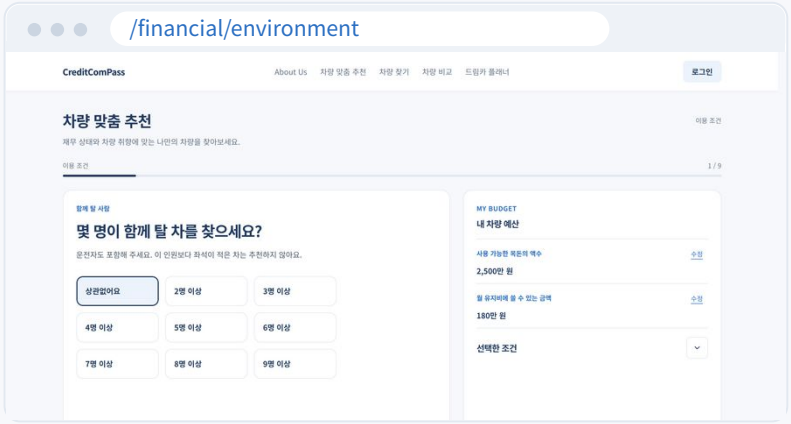
- ① 비슷한 상황의 프로필을 골라 시작합니다
- ② 재무 문항에 하나씩 답하고, 지출과 대출은 총액이나 상세 중 골라 입력합니다
- ③ 오른쪽에서 내 차량 예산을 바로 확인합니다
- 이용 환경과 차량 취향을 따로 묻고, 마지막에 답변을 고칠 수 있습니다



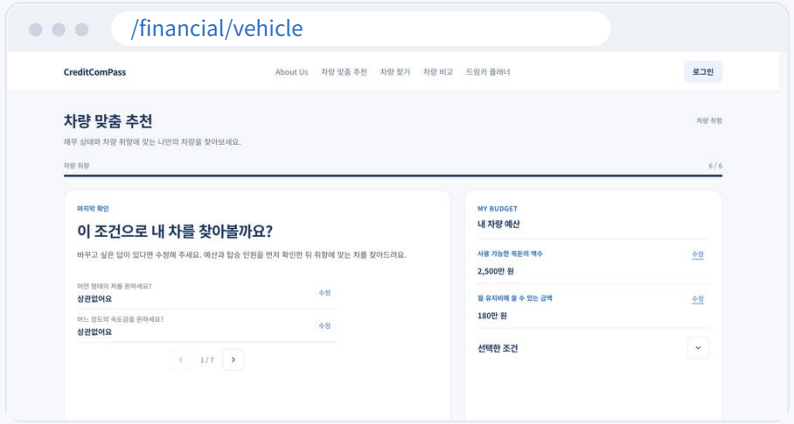

"당신과 가장 가까운 상황은?"




재무 문답과 내 차량 예산




이용 조건 (9문항)




차량 취향, 답변 확인과 수정

# 주요 기능


01 팀 소개
팀원과 맡은 역할

02 프로젝트 소개 및 기획 의도
사용자 문제, 기획 의도, 요구사항, 화면 변화

03 주요 기능 실습
진단, 추천, 비교, 플래너, 콤파, 운영

04 시스템 아키텍처
React, Spring Boot, FastAPI 구성과 저장 위치

05 데이터 파이프라인
수집 → 정제 → DB 적재

06 데이터 분석 및 모델링
데이터 탐색, 모델 구조, 평가, 후속 실험

07 ERD
차량 원본과 실제 저장 구조

08 보안 적용 내용
로그인, 권한, 입력 검증, 자동 가입 방지

09 협업 방식
브랜치 전략과 일정 관리

10 트러블슈팅
주요 문제 해결 사례

11 향후 개선 방향
개발 결과, 한계, 다음 과제


▶ 실습 사이트
d17d1rs6mihtjx.cloudfront.net

# 주요 기능을 한눈에 보고 직접 써 보세요

## 1 재무 진단

소득, 지출, 상환액, 목돈을 답하면 차량 예산을 알려줍니다

/financial

▶ 실습하기

## 2 차량 추천

재무, 이용 환경, 취향에 맞는 차량 순위와 이유를 보여줍니다

/financial/recommendations

▶ 실습하기

## 3 차량 탐색과 비교

월 비용까지 최대 3대를 비교하고 구매 조건을 바꿔 봅니다

/vehicles

▶ 실습하기

## 4 드림카 플래너

원하는 차를 사기 위한 저축 계획과 진행 상황을 봅니다

/dream-car

▶ 실습하기

## 5 취향 테스트와 콤파

8개 문항 취향 테스트, AI 도우미, 1:1 상담

/taste-test

▶ 실습하기

## 6 관리자 콘솔

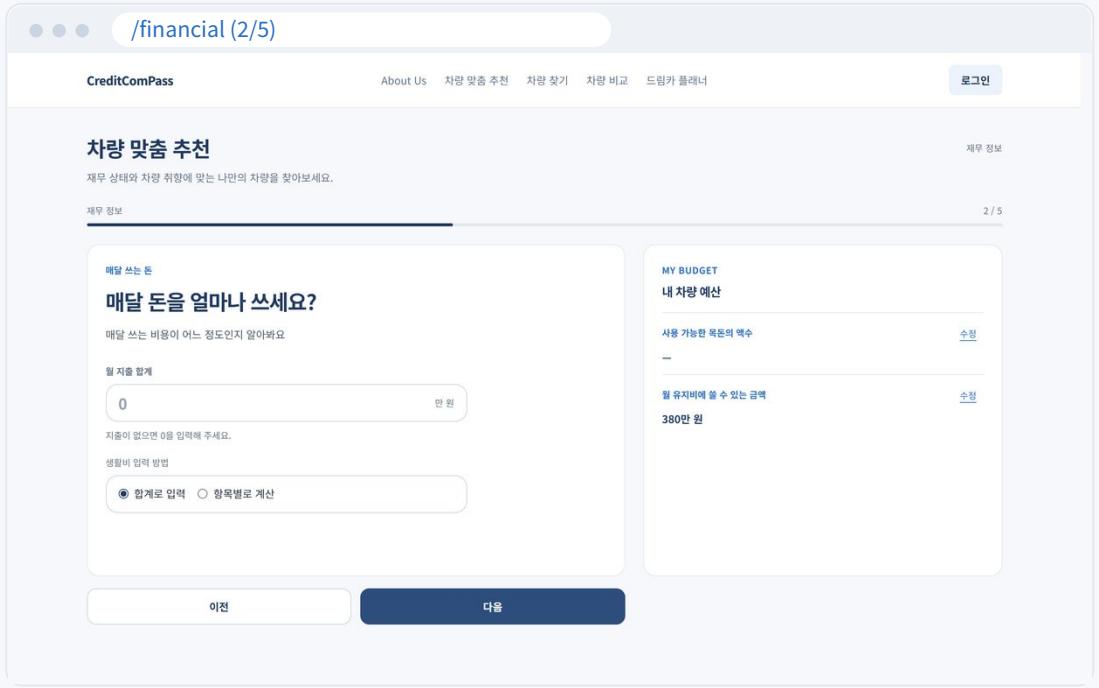
차량, 모델, 회원, 상담을 관리합니다

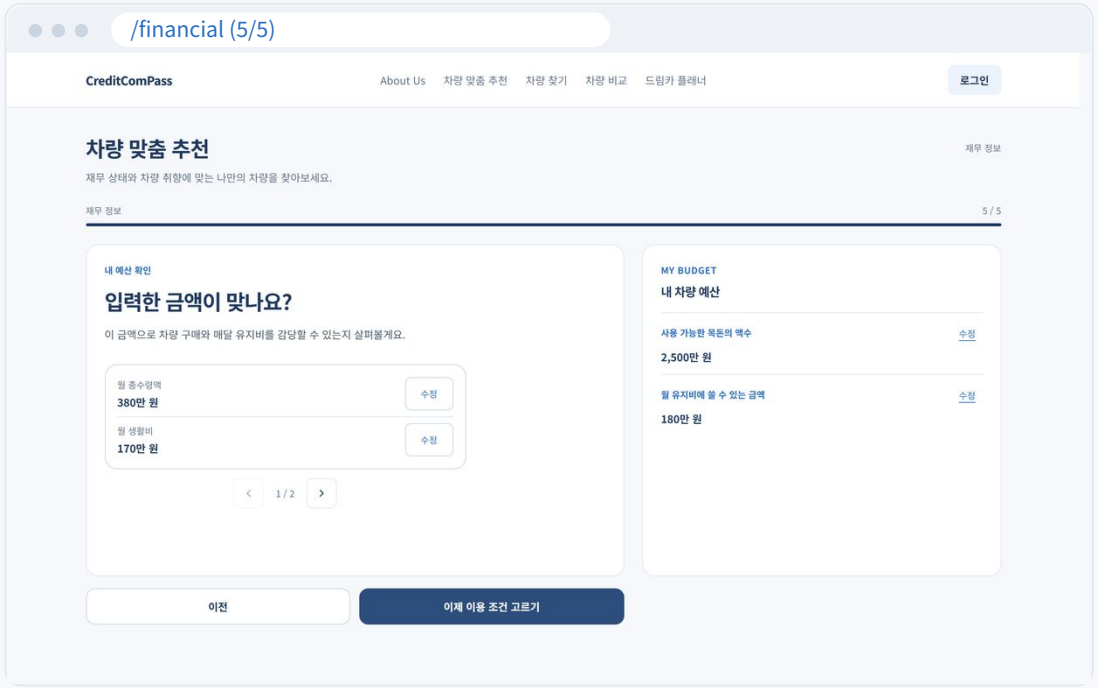
/console

▶ 실습하기

▶ 실습하기

# 재무 진단과 예산 요약




**1** 문항에 하나씩 답하면 오른쪽에 예산이 나옵니다

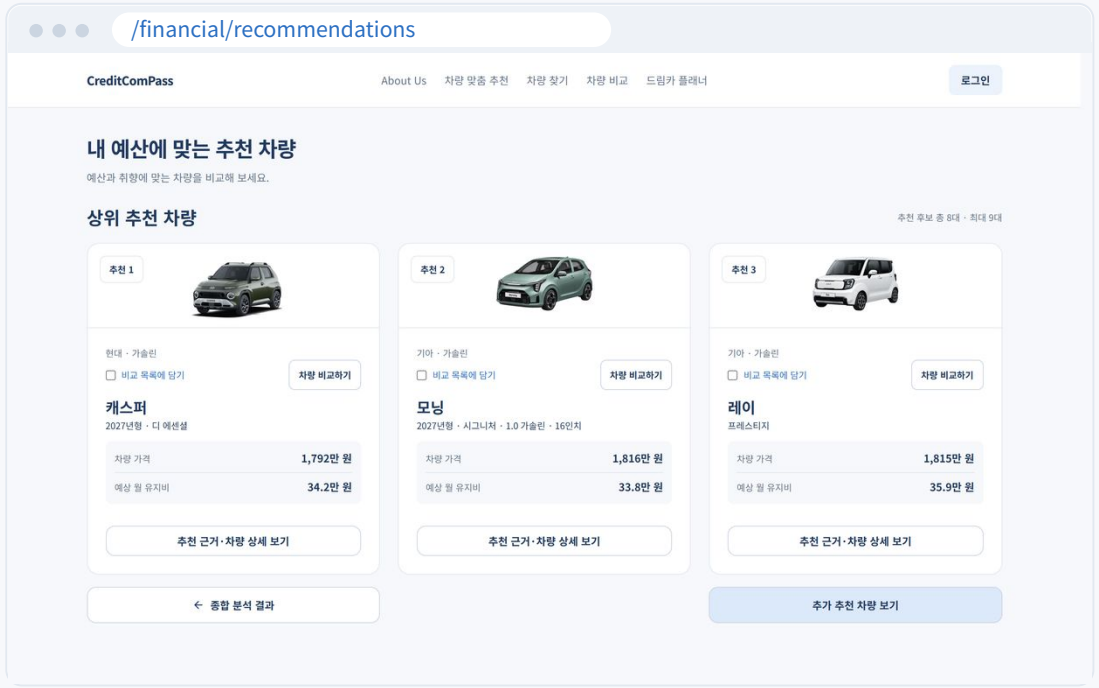
**2** 입력 확인과 MY BUDGET 요약

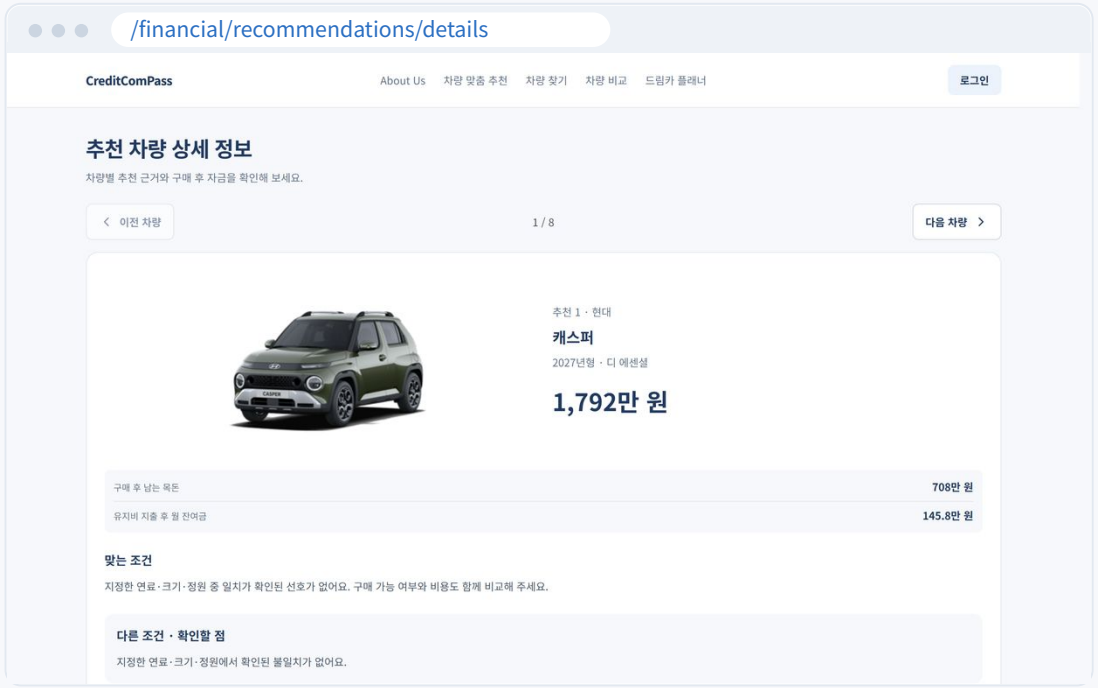
- 월 총수령액, 지출, 대출 상환액, 구매에 쓸 목돈을 입력합니다
- 지출과 대출은 합계나 상세 중 골라 입력합니다
- 재무 확인이 끝나면 이용 환경과 취향 질문으로 넘어갑니다

**월 유지비 가용액**
**소득 – 생활 지출 – 대출 월 상환액**

▶ 실습하기

# 추천과 추천 근거




**1** 추천 후보, 차량 가격, 예상 월 유지비

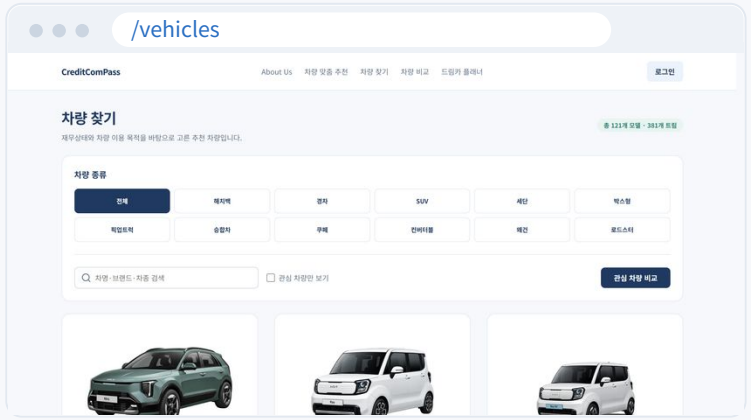
**2** 추천 근거, 맞는 조건, 확인할 점

- 재무, 이용 환경, 취향 조건에 맞는 후보와 추천 이유를 보여줍니다
- 차량 상세, 비교 추가, 취향 수정으로 바로 이어집니다
- 최대 9대를 요청하고, 조건에 맞는 차가 적으면 그만큼만 보여줍니다

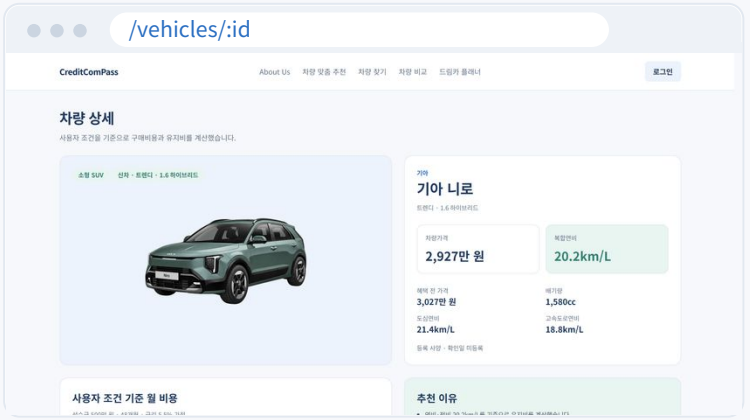
**점수는 순위를 정하기 위한 값이며, 만족도나 구매 확률이 아닙니다**

▶ 실습하기

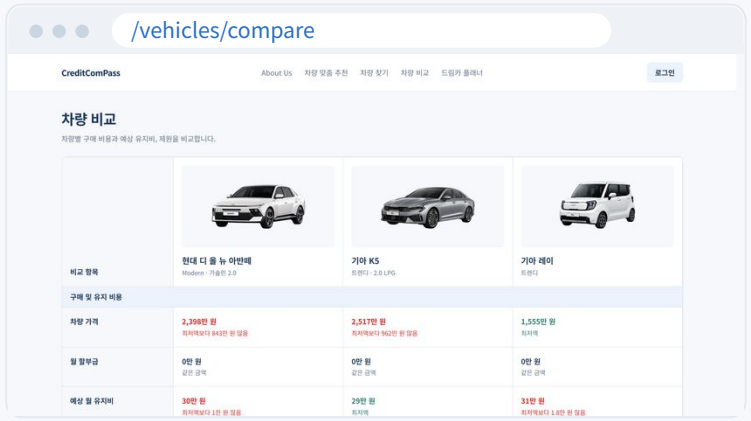
# 차량 탐색, 상세, 비교, 시뮬레이션



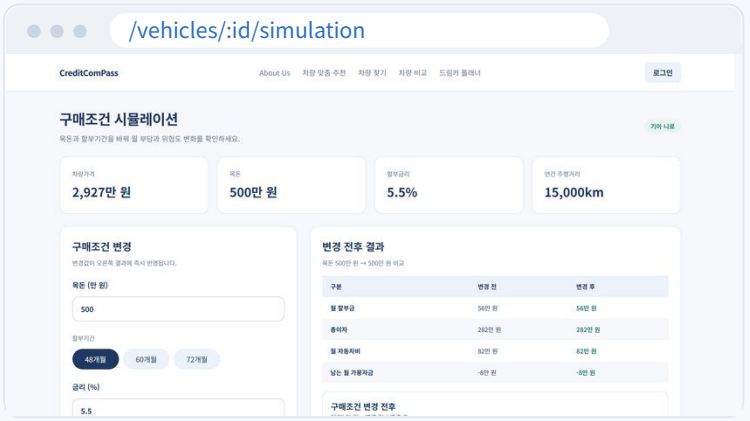

1 진단 없이 차량 찾기




2 제원과 비용 확인




3 최대 3대를 같은 조건으로 비교




4 조건 변경 전후 금액

구매비용

**차량 가격과 월 할부금**

월 유지비

**보험료, 연료비, 세금 등**

재무 적합도

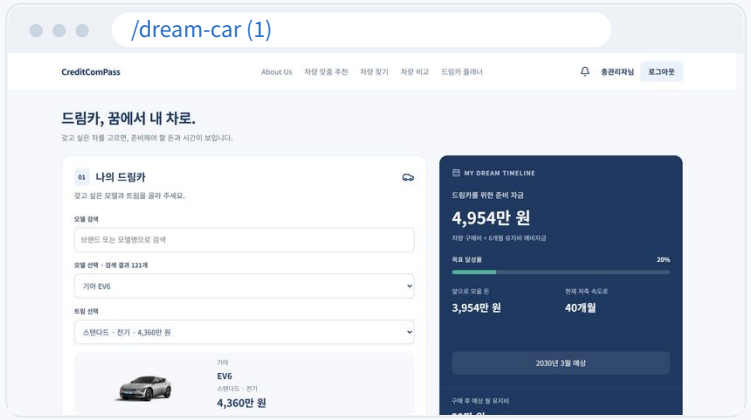
**차량비 지출 후 월 잔여금**

- 최대 3대를 같은 조건으로 비교
- 목돈, 할부 기간, 금리를 바꿨을 때 금액 확인

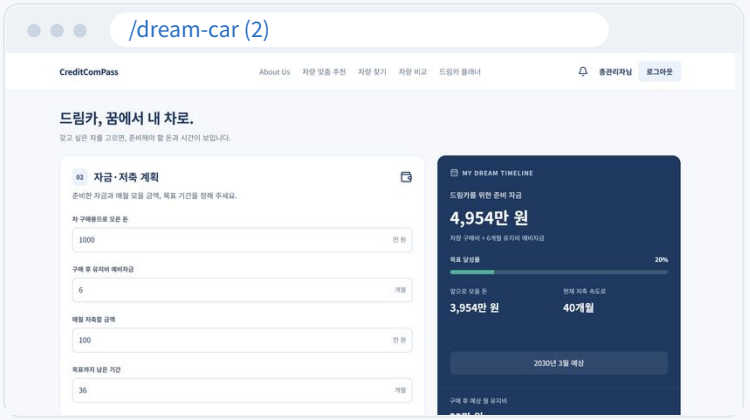
금리와 연료비는 계산을 위한 가정이며, 실제 대출 조건과 다를 수 있습니다

▶ 실습하기

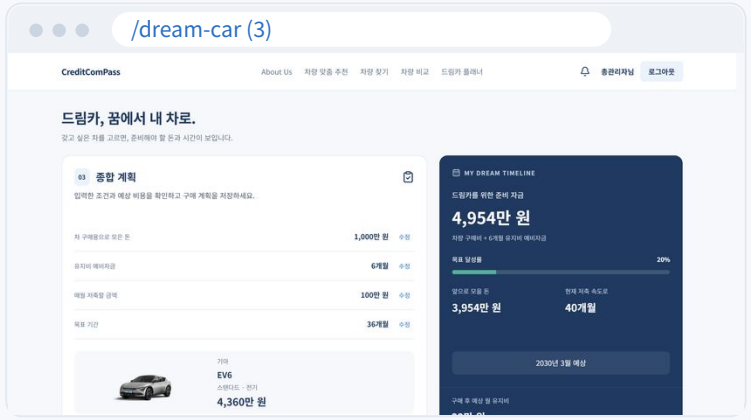
# 드림카 플래너와 저축 진행




**1** 나의 드림카 모델과 트림




**2** 자금과 저축 계획




**3** 종합 계획




**4** 저축 실행 현황

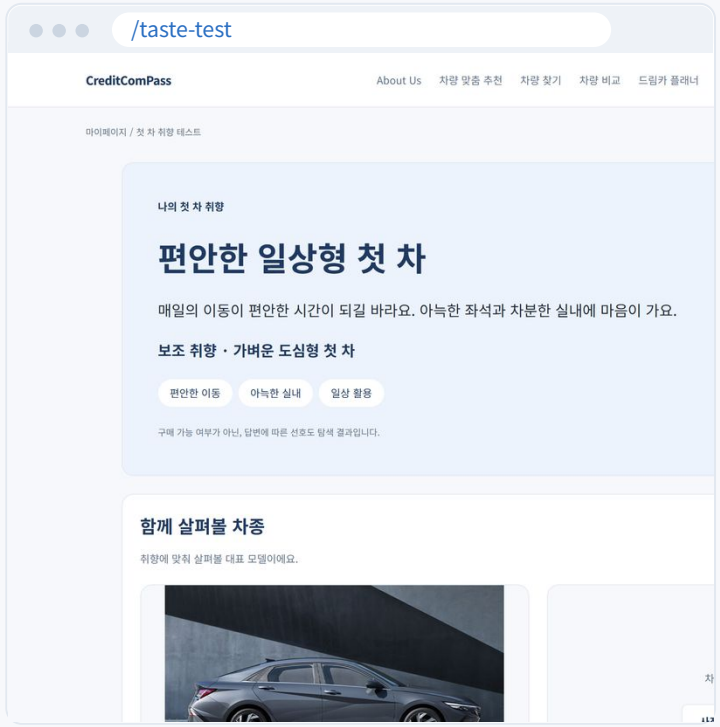
## 원하는 차를 먼저 고르고, 자금과 시간을 역산

- 현재 준비금, 월 저축액, 목표 기간
- 준비 자금, 앞으로 모을 돈, 예상 기간, 월 유지비
- 구매 계획과 저축 진행 상황을 함께 봅니다
- 목표 달성과 정기 기록 알림

예상 시점은 입력값으로 계산한 것이며, 실제 입금이나 자동이체는 하지 않습니다

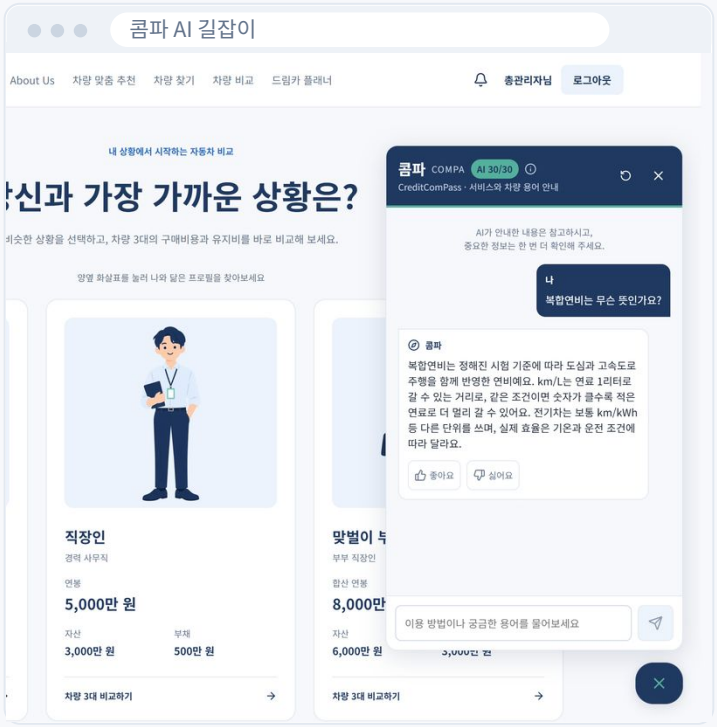
▶ 실습하기

# 취향 테스트, 콤파, 1:1 상담




## 취향 테스트

8개 문항에 답하면 차량 취향 유형을 알려줍니다




## 콤파

용어와 추천 질문은 정해진 답을 보여주고, 직접 입력한 질문만 문서를 찾아 외부 LLM으로 답합니다




## 1:1 상담

운영자가 회원 질문에 실시간으로 답합니다.
WebSocket 기반이며 콤파와는 별개입니다

▶ 실습하기

# 관리자와 모델 운영




운영 현황과 점검 사항




모델 평가 기록과 트래픽 분배

**차량 관리**
등록, 제원, 가격, 공개 여부

**추천 관리**
추천 후보와 제원 확인 상태

**회원과 권한**
회원 상태, 역할, 화면 권한

**상담과 콤파**
1:1 상담, 대화, 평가

**운영 이력**
프롬프트 버전과 운영 로그

- 사용자 화면과 /console 운영 화면을 나눴습니다
- 일반 관리자와 총관리자의 화면 권한을 확인합니다
- 실행 중이고 입력과 맞는 모델에만 트래픽을 나눕니다
- 차량을 바꾸면 검색 인덱스와 모델 카탈로그에도 영향이 갑니다

# 시스템 아키텍처


01 팀 소개
팀원과 맡은 역할

02 프로젝트 소개 및 기획 의도
사용자 문제, 기획 의도, 요구사항, 화면 변화

03 주요 기능 실습
진단, 추천, 비교, 플래너, 콤파, 운영

04 시스템 아키텍처
React, Spring Boot, FastAPI 구성과 저장 위치

05 데이터 파이프라인
수집 → 정제 → DB 적재

06 데이터 분석 및 모델링
데이터 탐색, 모델 구조, 평가, 후속 실험

07 ERD
차량 원본과 실제 저장 구조

08 보안 적용 내용
로그인, 권한, 입력 검증, 자동 가입 방지

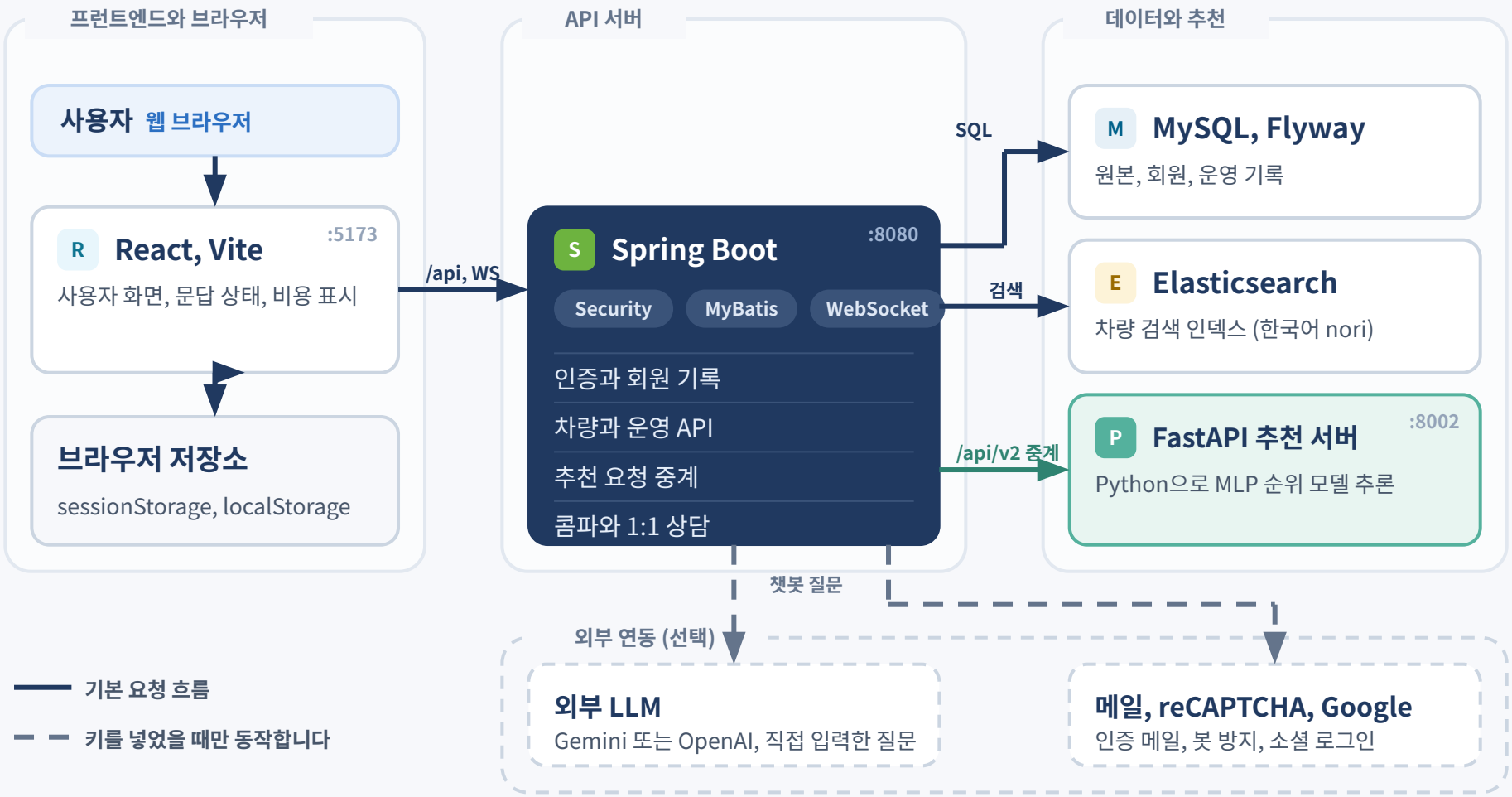
09 협업 방식
브랜치 전략과 일정 관리

10 트러블슈팅
주요 문제 해결 사례

11 향후 개선 방향
개발 결과, 한계, 다음 과제


**▶ 실습 사이트**
d17d1rs6mihtjx.cloudfront.net

# 시스템 아키텍처와 기술 선택



# 추천 모델에 필요한 데이터

## 실제 차량 정보

**제조사 가격표와 제원 자료**

가격, 연료, 크기, 좌석 수 등을 정리합니다.
어떤 차량을 추천할 수 있는지 결정합니다.

## 가상 사용자와 선택 기록

**직접 만든 더미 데이터 생성기**

소득·자산·취향이 다른 사용자를 만들고,
차량 두 대 중 하나를 고르는 기록을 만듭니다.

**차량 정보** + **가상 선택 기록**

**이 두 데이터를 연결해, 조건에 맞는 차량의 추천 순서를 학습합니다**

# 더미 데이터 생성기를 만든 이유

## 실제 고객의 차량 선택 기록을 확보하지 못했습니다

예산과 취향이 달라질 때
어떤 차량을 고르는지 학습할 자료가
필요했습니다.

## 그래서 생성기를 직접 만들었습니다

01
### 가상 사용자 생성
재무 조건과 취향 설정

02
### 차량 선택 기록 생성
조건에 맞는 두 차량 비교

같은 방식으로 반복 생성하고,
예산·좌석·중복 여부를 검사했습니다.

**합성 데이터 = 규칙과 가정으로 만든 학습 자료. 실제 구매 기록과 구분합니다**

# 가상 사용자를 만드는 방법

**30개**
처음 작성한 생활·재무 사례

**39개**
추가 사례 9개를 반영

**생활 상황의 폭 확대**

---

01
### 사례를 고릅니다
생활 배경과 재무 조건을 출발점으로 사용

02
### 조건을 변형합니다
소득·지출·자산과 취향·중요도를 조정

03
### 조건을 검사합니다
구매·유지비·좌석 조건을 충족하는지 확인

**원하는 차량이 나오도록 사용자 재무를 거꾸로 맞추지 않습니다**

# 차량 두 대의 선택 데이터를 만드는 방법

**설명용 사례** **5명이 탈 차가 필요하고, 유지비를 중요하게 보는 사용자**

**먼저 확인** 예산과 유지비를 감당할 수 있고, 5명이 탈 수 있는 차량만 비교

### 차량 A

유지비 부담이 더 작음
가속 성능은 상대적으로 낮음

**비교**

### 차량 B

가속 성능이 더 좋음
유지비 부담은 상대적으로 큼

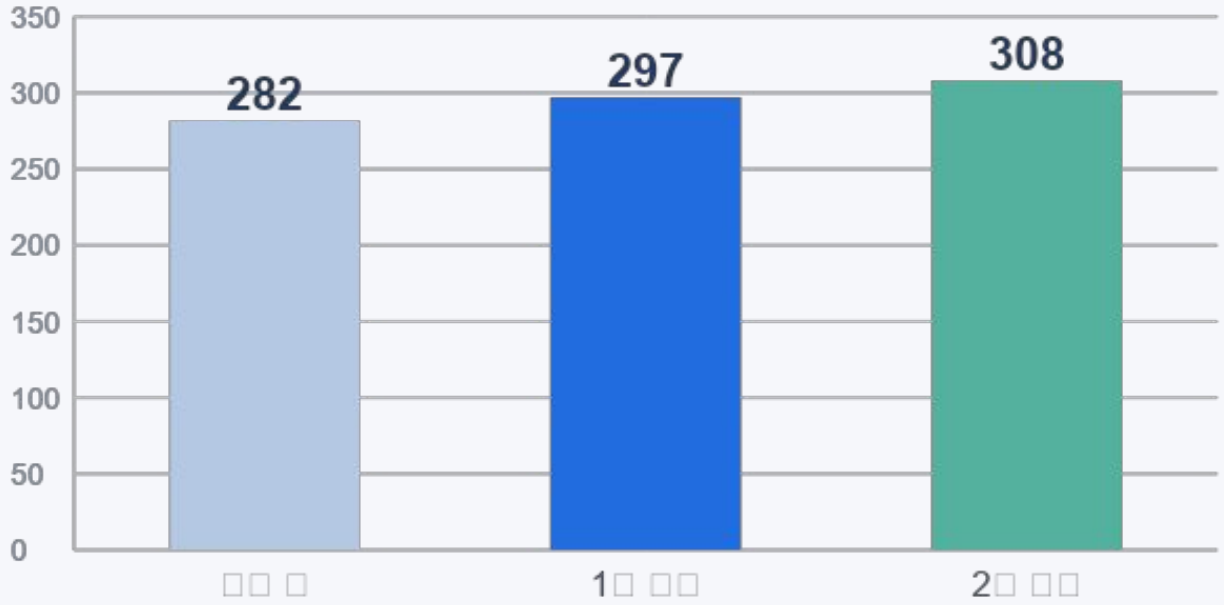
→ **선택 기록**
**A 또는 B**

**취향과 중요도에 따라 선택 확률을 만들고, 추첨 결과를 학습 정답으로 저장합니다**

같은 조건에서도 항상 같은 차를 선택하도록 만들지는 않았습니다

# 부족한 차량 정보를 추가한 과정

## 10/27, 공개 추천 차량의 사양 수




사양: 같은 모델 안에서 가격·좌석 등이 다른 세부 구성

### 1차 보충: 15사양 추가

해치백, 픽업, 다인승 차량 등
부족한 차량 종류를 보충했습니다.

### 2차 보충: 11사양 추가

고가 SUV·픽업 등을 더해
비어 있던 가격대와 조건을 채웠습니다.

**차량 목록을 늘린 단계입니다. 이때 모델을 다시 학습하지는 않았습니다**

# 화면에서 묻는 취향을 데이터에 반영

10/30

## 사용자가 화면에서 고르는 항목

차량 윤곽과 가속 성능
연료와 차량 크기
비용·자금 보존·유지비의 중요도

**“상관없음”과 중요도 차이도 포함**

### 입력과 선택 정답을 다시 생성

**10,000명**
가상 사용자

**99,442건**
차량 비교 선택

96개 모델 · 308개 사양을 사용

**화면의 취향 입력과 학습 데이터가 맞도록 바꾸고, 모델도 다시 학습했습니다**

# 더 다양한 조건으로 학습 데이터 확대

## 11/2

### 추가한 차량과 조건

고가 차량을 추가하고,
차량 높이 정보도 입력에 반영했습니다.

높은 자산 구간의 가상 사용자도 생성해
새 차량이 비교에 등장하도록 했습니다.

**347개**
차량 사양

**39개**
모델 입력 특성

**300만 쌍**
새로 생성한 비교 선택 데이터

**확장한 차량 목록과 조건으로 데이터를 새로 만들고, 모델을 다시 학습했습니다**

# 서비스와 학습의 차량 목록 통합

11/3

## 공통 차량 목록으로 재생성·재학습

**384사양**

전체 관리 목록

→

**381사양**

공개 차량만 학습·추천에 사용

검수 대기 3사양 제외

11/4

## 누락 제원 보완

전장 39건 · 좌석 25건을 제조사 근거로 보완

**11/4에는 제원만 보완했습니다. 11/3에 학습한 모델 가중치를 그대로 사용합니다**

MODEL TERMS

# 모델 용어를 먼저 이해하기

**특성 (Feature)**

**모델이 판단할 때 참고하는 숫자**

예: 차량 가격, 좌석 수, 사용자의 취향 중요도

**가중치 (Weight)**

**각 입력이 점수에 미치는 영향을 정하는 값**

학습 과정에서 데이터를 보고 값을 조정합니다.

**학습 (Training)**

**선택 기록을 보고 점수 계산 방식을 조정하는 과정**

A를 고른 기록이면 A의 점수가 높아지도록 조정합니다.

**추론 (Inference)**

**학습을 마친 모델로 새 차량의 점수를 계산하는 과정**

서비스에서는 조건을 통과한 후보의 추천 순서를 정합니다.

# 모델이 추천 순서를 배우는 방법

**같은 사용자**

**입력 A**

**사용자 + 차량 A**

재무·취향·차량 특성

**입력 B**

**사용자 + 차량 B**

같은 사용자, 다른 차량

→

**MLP로 점수 계산**

차량마다
점수 1개를 계산

A와 B에 같은 모델 사용

→

**점수 차이를 정답과 비교**

A를 고른 기록이라면
A 점수가 더 높아지도록
모델을 조정합니다.

**쌍별 학습: 차량 두 대의 점수를 비교해, 선택한 차의 점수가 높아지도록 조정**

# 선형 모델과 MLP의 차이

## 선형 모델

**입력별 영향을 더해 점수 계산**

각 입력 × 가중치의 합 + 기본값

같은 입력의 영향은 다른 조건이 바뀌어도
일정합니다. 입력에 이미 조합한 특성이
있다면 그 조합의 영향도 반영할 수 있습니다.

## MLP · 다층 퍼셉트론

**여러 계산층으로 조건의 조합을 학습**

입력 39개 / 중간층 64개·32개 / 점수 1개

중간층은 입력을 섞어 계산하는 단계입니다.
음수는 0으로 바꾸는 계산(ReLU)을 넣어
조건에 따라 달라지는 영향을 표현합니다.

**설명용 예: 같은 유지비 차이라도, 사용자가 유지비를 얼마나 중요하게 보는지 함께 고려**

**복잡한 구조가 항상 더 정확하지는 않습니다. 같은 데이터로 비교해야 합니다.**

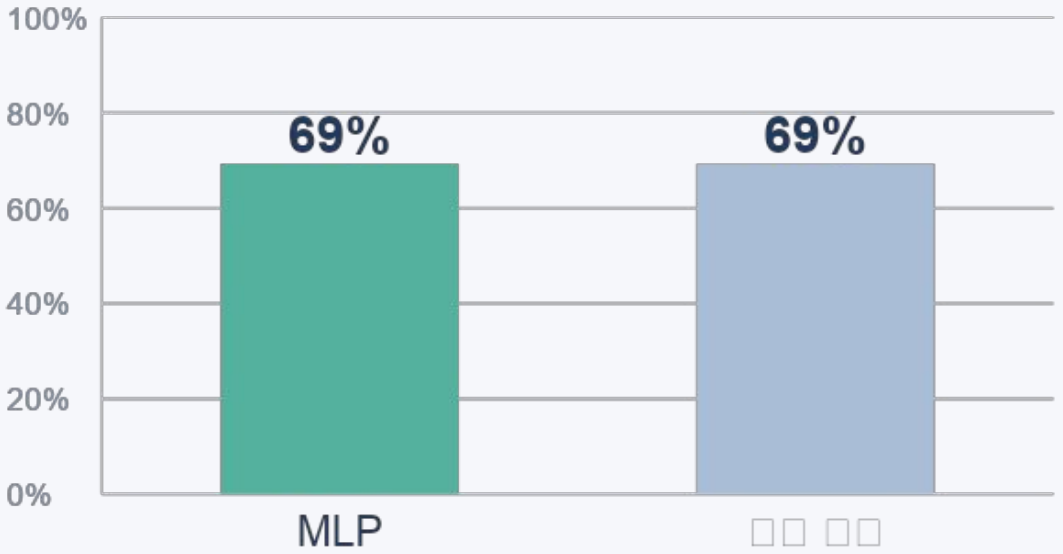
# 모델을 확인한 방법과 평가 결과

## 자료를 나눠서 확인했습니다

| | |
|---|---|
| 약 70% | **학습** 모델 점수 조정 |
| 약 15% | **검증** 사용할 모델 선택 |
| 약 15% | **평가** 마지막 성능 확인 |

같은 재무 그룹은 서로 다른 묶음에 넣지 않았습니다

## 합성 차량 선택을 맞힌 비율




평가 비교쌍 449,959개 · 동일 자료로 비교

**두 모델의 결과는 비슷합니다. 이 수치는 실제 구매 정확도나 만족도를 뜻하지 않습니다**

READING THE RESULTS

# 69%라는 평가 수치의 의미

**평가한 질문** **“가상 사용자가 고른 차량의 점수를 더 높게 줬는가?”**

## 선형 모델 69.38%  MLP 69.31%

차이는 0.1%p 미만입니다. 현재 평가만으로 MLP가 더 낫다고 말할 수 없습니다.

**현재 확인한 것** 합성 선택 기록에서 두 모델이 비슷한 성능을 보였습니다.

**다음에 확인할 것** 실제 사용자의 선택과 만족도에서도 도움이 되는지 검증합니다.

**69%는 실제 구매 확률이나 사용자 만족도를 뜻하지 않습니다.**

# 현재 추천 모델 한눈에 보기

1 **입력**
재무와 취향

2 **조건 확인**
재무·필수 좌석

3 **가격대 적용**
구매 가능액 기준

4 **모델 순위**
취향 점수로 정렬

5 **추천 결과**
차량과 비용

**396,746명**
가상 사용자

**300만 쌍**
비교 선택

**121모델**
공개 381사양

**39개**
모델 입력 특성

## 합성 선택 평가 정확도 69.31%

실제 구매·만족도 검증은 후속 과제

**저장소 기본 실행 설정: 11/3 학습 가중치 + 11/4 제원 보완**

# ERD


01 팀 소개
팀원과 맡은 역할

02 프로젝트 소개 및 기획 의도
사용자 문제, 기획 의도, 요구사항, 화면 변화

03 주요 기능 실습
진단, 추천, 비교, 플래너, 콤파, 운영

04 시스템 아키텍처
React, Spring Boot, FastAPI 구성과 저장 위치

05 데이터 파이프라인
수집 → 정제 → DB 적재

06 데이터 분석 및 모델링
데이터 탐색, 모델 구조, 평가, 후속 실험

07 ERD
차량 원본과 실제 저장 구조

08 보안 적용 내용
로그인, 권한, 입력 검증, 자동 가입 방지

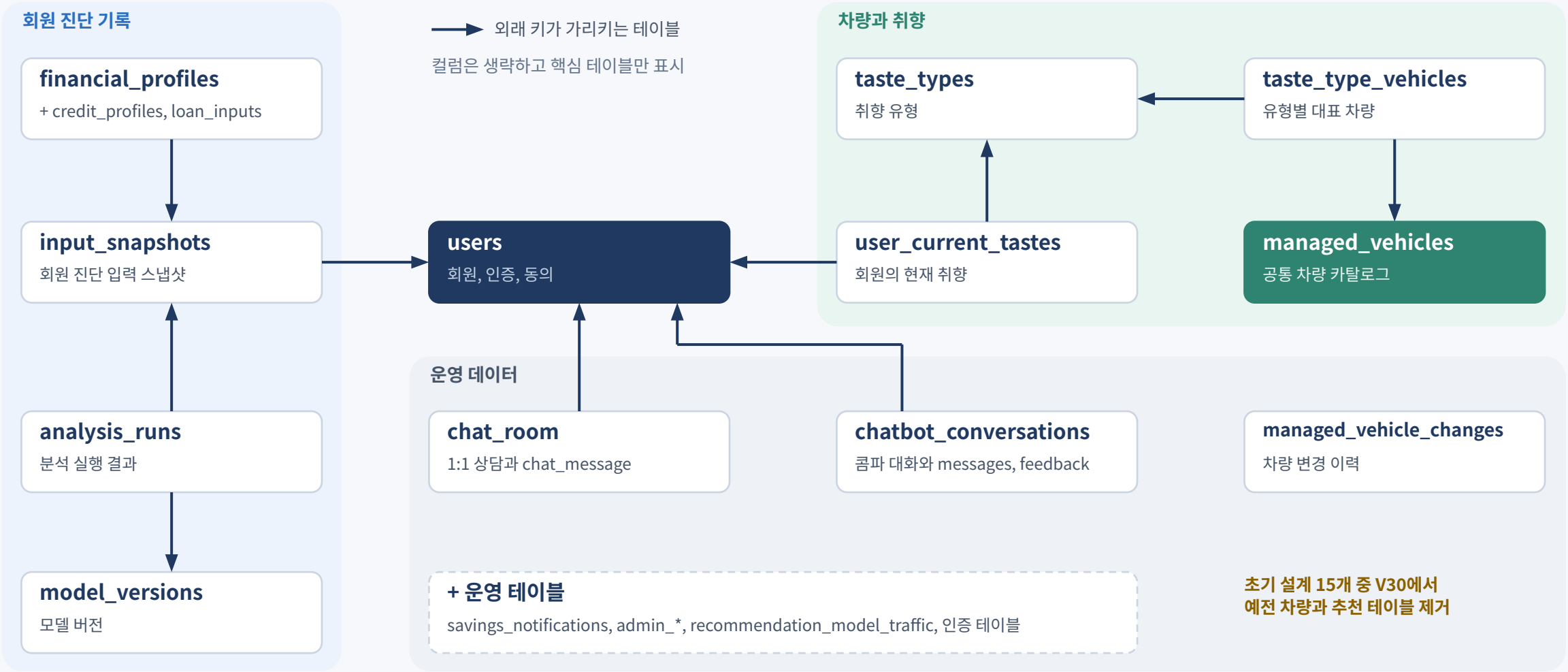
09 협업 방식
브랜치 전략과 일정 관리

10 트러블슈팅
주요 문제 해결 사례

11 향후 개선 방향
개발 결과, 한계, 다음 과제


▶ 실습 사이트
d17d1rs6mihtjx.cloudfront.net

# ERD로 본 차량 원본과 실제 저장 구조



# 보안 적용 내용


01 팀 소개
팀원과 맡은 역할

02 프로젝트 소개 및 기획 의도
사용자 문제, 기획 의도, 요구사항, 화면 변화

03 주요 기능 실습
진단, 추천, 비교, 플래너, 콤파, 운영

04 시스템 아키텍처
React, Spring Boot, FastAPI 구성과 저장 위치

05 데이터 파이프라인
수집 → 정제 → DB 적재

06 데이터 분석 및 모델링
데이터 탐색, 모델 구조, 평가, 후속 실험

07 ERD
차량 원본과 실제 저장 구조

08 보안 적용 내용
로그인, 권한, 입력 검증, 자동 가입 방지

09 협업 방식
브랜치 전략과 일정 관리

10 트러블슈팅
주요 문제 해결 사례

11 향후 개선 방향
개발 결과, 한계, 다음 과제


▶ 실습 사이트
d17d1rs6mihtjx.cloudfront.net

# 보안은 이렇게 적용했습니다

## 로그인과 토큰

- 비밀번호는 BCrypt로 해시해서 저장합니다
- Access 토큰은 30분짜리이고 브라우저 메모리에만 둡니다
- Refresh 토큰은 14일이며 HttpOnly, SameSite=Strict 쿠키에 담습니다
- 토큰을 재발급할 때마다 새 토큰으로 바꾸고, 로그아웃하면 DB에서 지웁니다
- 한 계정은 기기 5대까지 로그인할 수 있습니다

## 권한 나누기

- 서버에 세션을 두지 않는 STATELESS 방식입니다
- 공개, 로그인 회원, 화면별 관리자, 최고 관리자로 나눴습니다
- 관리자 권한은 요청할 때마다 DB에서 다시 확인합니다
- 일반 회원에게는 관리자 API가 없는 것처럼 404를 돌려줍니다
- /console 화면과 WebSocket 연결도 로그인 토큰을 확인합니다

## 입력 검증

- 서버는 @Valid로 요청 값을 검사합니다 (19개 파일)
- 비밀번호 규칙(8~20자, 문자 조합)은 서버와 화면이 같습니다
- 챗봇 질문은 500자까지 받고, 개인정보를 가린 뒤 LLM에 보냅니다
- SQL은 MyBatis #{} 바인딩만 씁니다 (${} 사용 0건)
- 화면에 HTML을 그대로 넣는 코드는 없습니다

## 자동 가입 방지와 키 관리

- 로그인을 3번 틀리면 reCAPTCHA, 5번 틀리면 10분 동안 잠급니다
- 이메일 인증 코드는 5분간 유효하고, 하루 300통까지 보냅니다
- 챗봇은 회원당 하루 30번까지 쓸 수 있습니다
- 비밀 키는 환경변수로 넣고, .env 파일은 git에 올리지 않습니다
- 추천 서버는 차량 카탈로그를 SHA-256으로 확인한 뒤 불러옵니다

**운영 전 확인** 개발용 기본값(JWT secret, 테스트용 reCAPTCHA 키)을 실제 값으로 바꾸고, 쿠키 Secure 옵션을 켜야 합니다

# 협업 방식


01 팀 소개
팀원과 맡은 역할

02 프로젝트 소개 및 기획 의도
사용자 문제, 기획 의도, 요구사항, 화면 변화

03 주요 기능 실습
진단, 추천, 비교, 플래너, 콤파, 운영

04 시스템 아키텍처
React, Spring Boot, FastAPI 구성과 저장 위치

05 데이터 파이프라인
수집 → 정제 → DB 적재

06 데이터 분석 및 모델링
데이터 탐색, 모델 구조, 평가, 후속 실험

07 ERD
차량 원본과 실제 저장 구조

08 보안 적용 내용
로그인, 권한, 입력 검증, 자동 가입 방지

09 협업 방식
브랜치 전략과 일정 관리

10 트러블슈팅
주요 문제 해결 사례

11 향후 개선 방향
개발 결과, 한계, 다음 과제


▶ 실습 사이트
d17d1rs6mihtjx.cloudfront.net

# Git 브랜치 전략과 협업 규칙

## 브랜치 흐름

**rsm** 류상민

**hhc** 하형철

**woj** 이원준

**feat/data** 지승렬

→ **develop**
합쳐서 확인

→ **main**
CI 통과 후 병합

- 각자 브랜치에서 작업한 뒤 develop에 합칩니다
- develop의 변경은 다시 각자 브랜치로 가져옵니다
- 문제가 없으면 develop을 main에 합칩니다
- main에 합친 뒤 main 코드로 다시 빌드합니다

## 커밋 메시지 규칙

- "type: 한국어 요약" 뒤에 무엇을 왜 바꿨는지 적습니다
- type은 feat, fix, docs 등 10가지입니다
- 293개 중 283개(96.6%)가 규칙을 지켰습니다
- feat 145, fix 79, style 28, docs 12개

## 병합 전 자동 검사

- push할 때마다 GitHub Actions CI가 돕니다
- 백엔드는 MySQL을 띄워 ./gradlew test를 돌립니다
- 프런트는 npm test를 돌립니다
- 두 검사를 통과해야 main에 합칠 수 있습니다

## AI 도구 공통 규칙

- AGENTS.md, CLAUDE.md, Cursor 규칙을 함께 둡니다
- 모두 같은 .agents/skills 문서를 따릅니다
- 테스트, UI, API, 이름, 설계, Git 기준을 담았습니다
- 누가 어떤 AI를 써도 같은 규칙으로 작업합니다

| 358개 | main 커밋 | 9/29 ~ 11/10 | 개발 기간 | 4명 | 참여 인원 | 65번 | 병합 (PR 5번) |
|---|---|---|---|---|---|---|---|

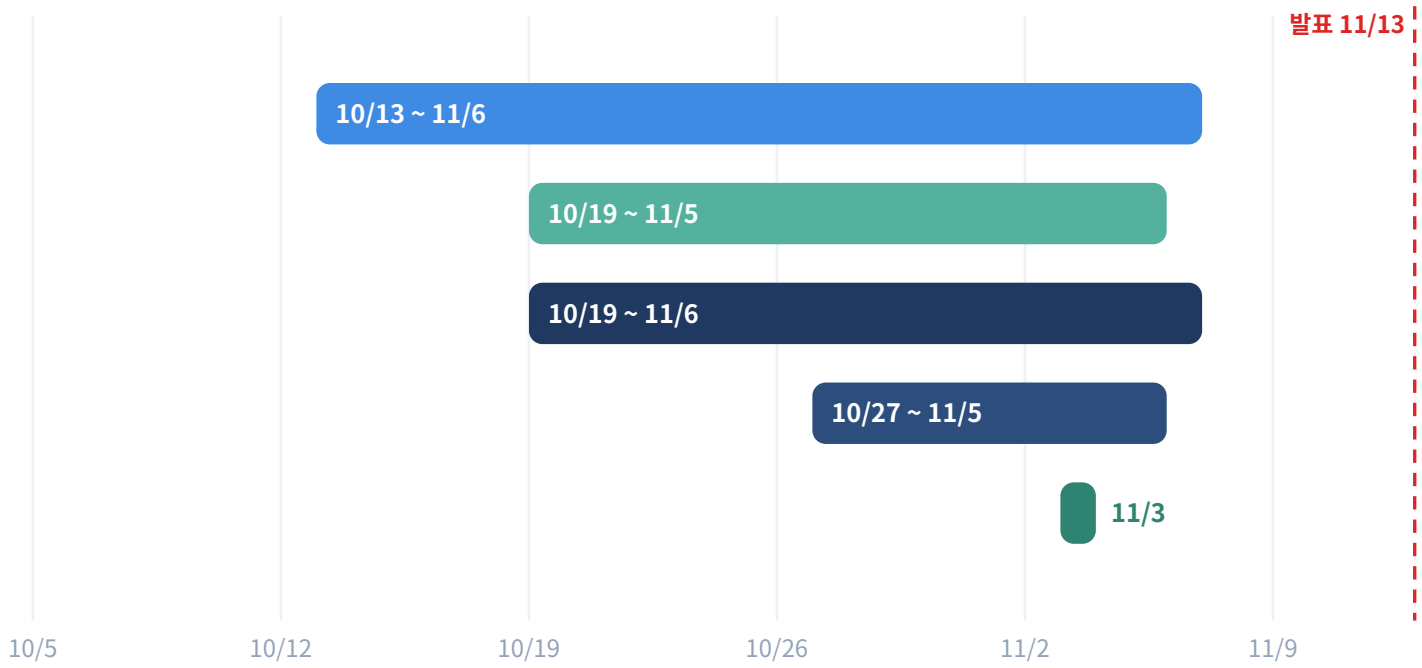
# 개발일정과 협업

## 초기 계획

기획발표 일정 10/1 ~ 11/13

요구사항, 데이터, 설계 | 진단과 추천 | 상세, 비교, 계획 | 통합, 평가, 발표

실제 주요 변화 (origin/main 첫 커밋부터 마지막 커밋까지)




**협업 기준**

API 계약 | 공통 차량 ID | 모델과 데이터 버전 | Flyway 마이그레이션

# 트러블슈팅


01 팀 소개
팀원과 맡은 역할

02 프로젝트 소개 및 기획 의도
사용자 문제, 기획 의도, 요구사항, 화면 변화

03 주요 기능 실습
진단, 추천, 비교, 플래너, 콤파, 운영

04 시스템 아키텍처
React, Spring Boot, FastAPI 구성과 저장 위치

05 데이터 파이프라인
수집 → 정제 → DB 적재

06 데이터 분석 및 모델링
데이터 탐색, 모델 구조, 평가, 후속 실험

07 ERD
차량 원본과 실제 저장 구조

08 보안 적용 내용
로그인, 권한, 입력 검증, 자동 가입 방지

09 협업 방식
브랜치 전략과 일정 관리

10 트러블슈팅
주요 문제 해결 사례

11 향후 개선 방향
개발 결과, 한계, 다음 과제


▶ 실습 사이트
d17d1rs6mihtjx.cloudfront.net

# 프로젝트를 진행하며 겪은 어려움

## 사례 A 팀원 공백 속 일정 유지

| | |
|---|---|
| 문제 | 예비군 훈련 등으로 승렬·형철이 빠지는 날이 많아 작업이 자주 끊겼습니다 |
| 대응 | 남은 팀원이 이어 맡고, 돌아오면 develop을 받아 바로 이어서 작업했습니다 |
| 공유 | 커밋에 무엇을 왜 바꿨는지 적어 빠진 동안의 변경을 이력으로 따라잡았습니다 |
| 결과 | 사람이 빠진 날이 많았지만 모두 잘 따라와 줘서 일정 안에 통합했습니다 |

## 사례 B AI 도구로 만든 결과물 검증

| | |
|---|---|
| 문제 | AI로 만든 코드와 데이터가 화면·DB·모델에 실제로 연동되는지 일일이 확인해야 했습니다 |
| 적용 | AGENTS.md와 공통 규칙 skill 7종으로 AI가 같은 기준을 따르게 하고, AI 로직은 코드 리뷰로 확인했습니다 |
| 규모 | 커밋 294개 중 199개(68%)를 AI와 함께 작성해서, 검토해야 할 리뷰가 계속 쌓였습니다 |
| 결과 | 리뷰와 CI 테스트를 통과한 코드만 main에 합쳤습니다 |

# 향후 개선 방향


01 팀 소개
팀원과 맡은 역할

02 프로젝트 소개 및 기획 의도
사용자 문제, 기획 의도, 요구사항, 화면 변화

03 주요 기능 실습
진단, 추천, 비교, 플래너, 콤파, 운영

04 시스템 아키텍처
React, Spring Boot, FastAPI 구성과 저장 위치

05 데이터 파이프라인
수집 → 정제 → DB 적재

06 데이터 분석 및 모델링
데이터 탐색, 모델 구조, 평가, 후속 실험

07 ERD
차량 원본과 실제 저장 구조

08 보안 적용 내용
로그인, 권한, 입력 검증, 자동 가입 방지

09 협업 방식
브랜치 전략과 일정 관리

10 트러블슈팅
주요 문제 해결 사례

11 향후 개선 방향
개발 결과, 한계, 다음 과제


▶ 실습 사이트
d17d1rs6mihtjx.cloudfront.net

# 개발 결과와 개선 방향

재무 입력 → 적격 후보와 추천 → 비용 비교 → 드림카 준비계획 → **회원 기록과 운영 관리**

**개발 결과**

- 재무 입력, 추천, 비교, 플래너를 하나의 흐름으로 이었습니다
- 차량 데이터와 추천 모델을 API로 연결했습니다
- 회원 기록과 운영 기능까지 갖췄습니다

**한계와 후속 방향**

- 실제 사용자의 비교 선택을 모아 배치로 재학습하고, 검증을 통과한 모델만 교체합니다
- MLP가 선형 모델보다 낫다는 결과는 아직 없습니다
- 브라우저에 저장한 계획과 저축 기록을 기기 간에 맞추기
- 차량 카탈로그가 바뀔 때 모델을 갱신하고 검증하는 절차 개선

## 감사합니다. 질문 받겠습니다